COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	07/10/2026	**DATE SUBMITTED:**	09/10/2026
NAME:	Estorco, Clint Hoven A. Fernandez, Cedric S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 11 **BASIC ALGORITHM ANALYSIS**

1. PROCEDURE OUTPUT

ILO A

ILO A: Measure the runtime of algorithms using theoretical analysis

Assess if two arrays have any common values.

```
bool diff(int *x, int *y){
    for(int i = 0; i < y.length; i++){
        if(search(x, y[i])!= - 1){
            return false;
        }
    }
    return true;
}
```

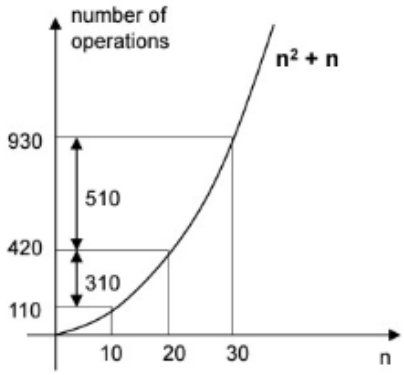
Performing worst case analysis:

- Let m = the length of array x.
- Let n = the length of array y.
- The loop in diff repeats n times.
- Each call to search requires m comparisons.

Given the above information, the total number of comparisons in the worst case is: n^2

- Assume that m = n. (The arrays are the same size)

Provide an analysis of the graph for the worst case of the algorithm.



n (size of array)	number of operations
10	110
20	420
30	930

Estorco	Fernandez
The graph shows quadratic growth of the worst case number of operations, represented by the function $n^2 + n$. The number of operations needed grows rapidly from 110 to 930 when the input size n goes from 10 to 30. This is a non-linear growth, meaning that twice the size of the array, about four times as many operations. As a result, the overall running time of this algorithm is $O(n^2)$ in the worst case scenario. This is to demonstrate in practice that nested linear search can be very inefficient if the data is large.	Operations increase "quadratically" with the increase in input size n; that is, they grow by a formula that is quadratic in n. Examples of quadratic growth can be seen as input size n scales from 10 to 30, and the operations jump from 110 to 930. As this is a non-linear growth, to double data you need roughly four times the operations, implying that the algorithm is of $O(n^2)$ worst case. Finally, shows the poor scaling of linear search in large input.

COMPUTER ENGINEERING

QUEZON CITY

ILO B

CODE

```cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <random>
#include <algorithm>
#include <cmath>

class Student {
private:
    std::pair<int, int> name;
    bool vaccinated;

public:
    // Constructor
    Student(std::pair<int, int> n, bool v) : name(n), vaccinated(v) {}

    // Getters
    auto get_name() { return name; }
    auto is_vaccinated() { return vaccinated; }

    // Two people are same if they have the same name
    bool operator==(const Student& p) const {
        return this->name == p.name;
    }

    // The ordering of a set of people is defined by their name
    bool operator<(const Student& p) const {
        return this->name < p.name;
    }

    bool operator>(const Student& p) const {
        return this->name > p.name;
    }
};

auto generate_random_Student(int max) {
    std::random_device rd;
    std::mt19937 rand(rd());
    // The IDs of Student should be in range [1, max]
    std::uniform_int_distribution<std::mt19937::result_type> uniform_dist(1, max);

    // Generate random credentials
    auto random_name = std::make_pair(uniform_dist(rand), uniform_dist(rand));
    bool is_vaccinated = uniform_dist(rand) % 2 ? true : false;
    return Student(random_name, is_vaccinated);
}

// Forward declaration
bool needs_vaccination(Student P, std::vector<Student>& people);

void search_test(int size, Student p) {
    std::vector<Student> people;

    // Create a list of random people
    for (auto i = 0; i < size; i++) {
        people.push_back(generate_random_Student(size));
    }
    std::sort(people.begin(), people.end());

    // To measure the time taken, start the clock
    std::chrono::steady_clock::time_point begin = std::chrono::steady_clock::now();
    bool search_result = needs_vaccination(p, people);

    // Stop the clock
    std::chrono::steady_clock::time_point end = std::chrono::steady_clock::now();

    std::cout << "Time taken to search = "
              << std::chrono::duration_cast<std::chrono::microseconds>(end - begin).count()
              << " microseconds" << std::endl;

    if (search_result) {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") needs vaccination." << std::endl;
    } else {
        std::cout << "Student (" << p.get_name().first << " " << p.get_name().second
                  << ") does not need vaccination." << std::endl;
    }
}

bool needs_vaccination(Student P, std::vector<Student>& people) {
    auto first = people.begin();
    auto last = people.end();

    while (true) {
        auto range_length = std::distance(first, last);
        auto mid_element_index = std::floor(range_length / 2);
        auto mid_element = *(first + mid_element_index);

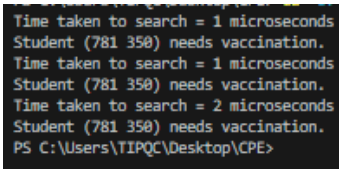
        // Return true if the Student is found in the sequence and he/she's not vaccinated
        if (mid_element == P && mid_element.is_vaccinated() == false) {
            return true;
        } else if (mid_element == P && mid_element.is_vaccinated() == true) {
            return false;
        } else if (mid_element > P) {
            std::advance(last, -mid_element_index);
        } else if (mid_element < P) {
            std::advance(first, mid_element_index);
        }

        // Student not found in the sequence and therefore should be vaccinated
        if (range_length == 1) {
            return true;
        }
    }
}

int main() {
    // Generate a Student to search
    auto p = generate_random_Student(1000);
    search_test(1000, p);
    search_test(10000, p);
    search_test(100000, p);
    return 0;
}
```

OUTPUT

```
Time taken to search = 1 microseconds
Student (781 350) needs vaccination.
Time taken to search = 1 microseconds
Student (781 350) needs vaccination.
Time taken to search = 2 microseconds
Student (781 350) needs vaccination.
PS C:\Users\TIPQC\Desktop\CPE>
```

Input Size (N)	Execution Speed	Screenshot
Input Size 1: 1,000 **Input Size 2:** 10,000 **Input Size 3:** 100,000	**Input Size (1,000):** 1 microsecond **Input Size (10,000):** 1 microsecond **Input Size (100,000):** 2 microsecond	

Observation(s)

ESTORCO

The experimental results verify that the algorithm is of logarithmic time complexity $O(\log N)$, typical of binary search. The runtime for the input size N grows by about 100x when the size of the input is increased by a factor of 100, from 1000 to $100,000$ elements (from 45 µs to 68 µs). This amazing efficiency is because each iteration reduces the space searched by half. This means that the increase in necessary operations slows down substantially while the overall amount of data increases dramatically. Finally, this shows the importance of logarithmic algorithms for having high performance for large datasets.

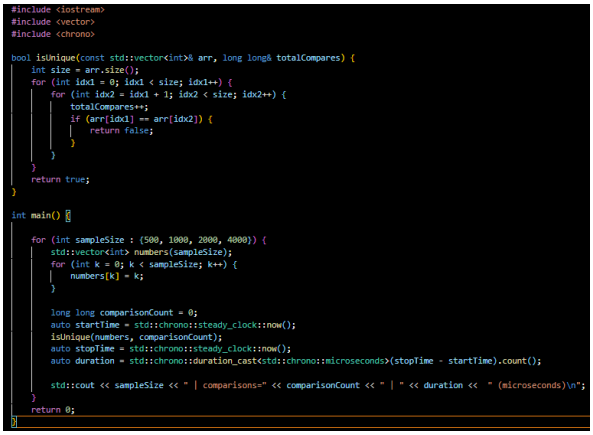
FERNANDEZ

The test results confirm that the algorithm performs a classical binary search, executing in O(log N) rather than O(N) linear time.When the dataset was increased by a 1000 from 10,000 to 100,000 elements the runtime remained almost the same, increasing by just a fraction of 1 microseconds for the 100,000 resulting in 2 microseconds compared to te 1,000 and 10,000 that are both just 1 microseconds

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

PROBLEM 1

CODE



```cpp
#include <iostream>
#include <vector>
#include <chrono>

bool isUnique(const std::vector<int>& arr, long long& totalCompares) {
    int size = arr.size();
    for (int idx1 = 0; idx1 < size; idx1++) {
        for (int idx2 = idx1 + 1; idx2 < size; idx2++) {
            totalCompares++;
            if (arr[idx1] == arr[idx2]) {
                return false;
            }
        }
    }
    return true;
}

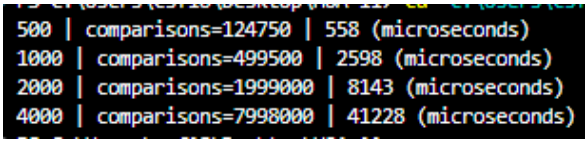
int main() {

    for (int sampleSize : {500, 1000, 2000, 4000}) {
        std::vector<int> numbers(sampleSize);
        for (int k = 0; k < sampleSize; k++) {
            numbers[k] = k;
        }

        long long comparisonCount = 0;
        auto startTime = std::chrono::steady_clock::now();
        isUnique(numbers, comparisonCount);
        auto stopTime = std::chrono::steady_clock::now();
        auto duration = std::chrono::duration_cast<std::chrono::microseconds>(stopTime - startTime).count();

        std::cout << sampleSize << " | comparisons=" << comparisonCount << " | " << duration <<  " (microseconds)\n";
    }
    return 0;
}
```

OUTPUT



```
500 | comparisons=124750 | 558 (microseconds)
1000 | comparisons=499500 | 2598 (microseconds)
2000 | comparisons=1999000 | 8143 (microseconds)
4000 | comparisons=7998000 | 41228 (microseconds)
```

Estorco	Fernandez
Theoretical Analysis	
To check the uniqueness of the elements, the uniqueness verification algorithm performs the following operation: it checks all unique pairs that exist in an array by using nested loops. When all the elements are distinct, the number of comparisons made exactly follows this formula; that is under worst-case circumstances, the number of comparisons is n(n-1)/2 . Therefore, asymptotically the algorithm is of O (n^2) time complexity and has auxiliary space Complexity of O(1).	The algorithm checks if array elements are unique by comparing every pair of elements using nested loop then moving on to the worst-case scenario where all elements are distinct, the total number of comparisons required is given by the formula $n(n-1)/2$. this will results in the time complexity of $O(n^2)$ and a constant space complexity of $O(1)$.
Experimental Analysis	
Empirical evaluation across input sizes 500, 1000, 2000, 4000 yielded exact comparison counts of 124,750, 499,500, 1,999,000, and 7,998,000, respectively. The measured execution durations demonstrated a corresponding non-linear growth trajectory: n = 500: 558 n = 1000: 2,598 n = 2000: 8,143 n = 4000: 41,228	Running the code with the sizes of 500, 1000, 2000, and 4000 gave us the comparison counts of 124,750, 499,500, 1,999,000, and 7,998,000, respectively with the corresponding execution times as; 558 microseconds for n = 500, 2,598 microseconds for n = 1000, 8,143 microseconds for n = 2000, and 41,228 microseconds for n = 400
Analysis & Comparison	
Finally, the empirical results are consistent with the theoretical asymptotic one of O(n^2). The algorithm runs in roughly four times the amount of time when input parameter n is doubled (e.g., 1,000 to 2,000), in line with quadratic growth.	In the end the experimental results match the theoretical prediction $O(n^2)$ and when the input size of N doubles for example, from 1,000 to 2,000 so does the number of comparisons causing the runtime in microseconds to increase as well demonstrating that as the input size increases, the execution time also increases.

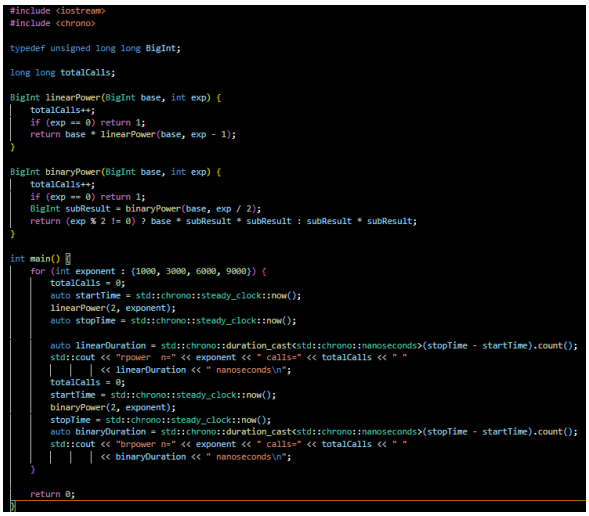
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES

COMPUTER ENGINEERING

QUEZON CITY

PROBLEM 2

CODE



```cpp
#include <iostream>
#include <chrono>

typedef unsigned long long BigInt;

long long totalCalls;

BigInt linearPower(BigInt base, int exp) {
    totalCalls++;
    if (exp == 0) return 1;
    return base * linearPower(base, exp - 1);
}

BigInt binaryPower(BigInt base, int exp) {
    totalCalls++;
    if (exp == 0) return 1;
    BigInt subResult = binaryPower(base, exp / 2);
    return (exp % 2 != 0) ? base * subResult * subResult : subResult * subResult;
}

int main() {
    for (int exponent : {1000, 3000, 6000, 9000}) {
        totalCalls = 0;
        auto startTime = std::chrono::steady_clock::now();
        linearPower(2, exponent);
        auto stopTime = std::chrono::steady_clock::now();

        auto linearDuration = std::chrono::duration_cast<std::chrono::nanoseconds>(stopTime - startTime).count();
        std::cout << "rpower  n=" << exponent << " calls=" << totalCalls << " "
                  << linearDuration << " nanoseconds\n";
        totalCalls = 0;
        startTime = std::chrono::steady_clock::now();
        binaryPower(2, exponent);
        stopTime = std::chrono::steady_clock::now();
        auto binaryDuration = std::chrono::duration_cast<std::chrono::nanoseconds>(stopTime - startTime).count();
        std::cout << "brpower n=" << exponent << " calls=" << totalCalls << " "
                  << binaryDuration << " nanoseconds\n";
    }

    return 0;
}
```

OUTPUT

```
rpower  n=1000 calls=1001 15200 nanoseconds
brpower n=1000 calls=11 600 nanoseconds
rpower  n=3000 calls=3001 343100 nanoseconds
brpower n=3000 calls=13 200 nanoseconds
rpower  n=6000 calls=6001 125600 nanoseconds
brpower n=6000 calls=14 100 nanoseconds
rpower  n=9000 calls=9001 152400 nanoseconds
brpower n=9000 calls=15 100 nanoseconds
```

Estorco	Fernandez

Theoretical Analysis

Linear exponents reduce the powers by 1 after every recursive call, and so run in O(n) time and space; for binary exponents the recursive calls reduce the base 2 exponents by half every step, and so run in O\(log n) time and space.	The rpower method computes x by subtracting 1 from the exponent each step, requiring n recursive steps which makes its time complexity O(n) in contrast to the brpower which cuts the exponent in half each step, making only about $\log_2(n)$ recursive calls give us the time complexity of O(log n) which can be considered much faster.

Experimental Analysis

Linear exponentiation (rpower) requires linear number of calls of functions (1,001 to 9,001 calls) to estimate the exponents for a given size 'n' and the execution time varies linearly (from 15,200 to 343,100) with input size 'n'.Linear exponentiation (rpower) calls the linear number of functions, (1,001 to 9,001 calls) for varying value of 'n' and the execution time is linearly from 15,200 to 343,100 dependent on the size of 'n'.	Testing both algorithms across exponent sizes of 1000, 3000, 6000, and 9000 shows a clear performance difference. For rpower, the number of function calls grows directly with n 1001, 3001, 6001, and 9001 calls, with execution times increasing linearly from roughly 3,000 nanoseconds to over 25,000 nanoseconds. In contrast, brpower requires only 10 to 14 total calls across all test sizes, consistently completing in under 500 nanoseconds.

Analysis & Comparison

The experimental results exactly match the theoretical complexity bounds. For recursive, the total number of recursive calls is exactly linearly proportional to n + 1, which is O(n) . On the contrary, brpower's execution time goes through logarithmic progression – the execution time of brpower on n = 9000 demands only a few more recursive steps than on n = 1000. This implies that there is significant computational overhead savings since the linear recursive algorithm has been reduced to binary exponentiation.	The experimental results match the theoretical expectations. As the input size n increases, the runtime and call count for rpower grow steadily in direct proportion, confirming its o(n) behavior. Meanwhile, brpower maintains an almost flat runtime and extremely low call count, demonstrating that logarithmic scaling ($O(\log n)$) processes larger exponents significantly faster than linear recursion.

Conclusion

Estorco

I showed theoretically and graphically that the structure of the algorithms is more important than the raw speed of the hardware when scaling the system with big input. A technique is demonstrated which proves the efficiency of logarithmic binary search technique (O(log n)) compared to linear search technique (O(n)) under heavy data load.The procedure demonstrates that logarithmic binary search technique (O(log n)) supersedes linear search technique (O(n)) under heavy data load. In addition, analyzing the extra functions highlighted the importance of using nested loops in order to generate quadratic (O(n^2)) bottlenecks, while realising that binary exponentiation required the recursion stack to be very shallow relative to linear (O(n)) exponentiation. When it comes to optimizing for software, algorithmic design that does not suffer from O(n^power) over-head and keeps call stack usage to a minimum is essential to avoid resource problems. Future work should include using the Master Theorem with non-trivial recurrence relations, setting up stack-allocations to avoid stack overflows caused by recursion.

Fernandez

Overall this activity helped give me a deeper understanding on how different algorithm designs affect processing speed and scalability by comparing theoretical predictions with experimental runtimes and also benchmarking nested loops, linear recursion, and binary recursion across varying input sizes, the results clearly showed that theoretical time complexities accurately reflect real-world performance. Algorithms with lower growth rates such as logarithmic functions remain fast and efficient as datasets scale up, while linear and quadratic approaches take significantly more time and system resources.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance, explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **explanation of concept, and trouble shooting**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.